Discover Artificial Intelligence

RESEARCH Open Access

De Pace: Optimizing initial guesses for homomorphic reciprocal and square root evaluation with CKKS

Simon Engel[1*], Lukas Horn[1], Thomas Prantl[1] and Samuel Kounev[1]

*Correspondence:
Simon Engel
simon.engel@uni-wuerzburg.de
[1]Computer Science Department, University of Wuerzburg, Sanderring 2, 97070 Wuerzburg, Bayern, Germany

Abstract

Homomorphic encryption allows computation on encrypted data, making it a promising technology for privacy-preserving machine learning. However, currently available fully homomorphic schemes, such as CKKS, only support addition and multiplication by default. Modern applications, such as neural networks, require more complex functions, such as division and square root. This raises a performance question for CKKS: What is the most efficient way to implement division and square root using only addition and multiplication? For CKKS, additional constraints apply: commonly used methods, such as comparisons and bit shifts, are no longer permitted, rendering most existing literature unusable. Our work addresses this challenge by analyzing and improving the performance of the method best suited for homomorphic computation of division and square root: the Newton–Raphson method. Since the primary degree of freedom in Newton–Raphson is the initial value, we formulate the determination of the parameters for a suitable linear helper function as a black-box optimization problem and analyze how different approaches to choosing these parameters affect the performance of homomorphic square root and division computation. Specifically, we evaluate four selection methods: (1) brute-force, (2) simulated annealing, (3) hill climbing, and (4) gradient descent. Our results show that the best optimization method finds parameters with lower RMSE and lower multiplicative depth than brute-force search, while reducing parameter generation time by multiple orders of magnitude. Our contribution is the problem formulation and its systematic evaluation over 603 000 configurations, providing a practical reference on the accuracy and multiplicative depth that can be expected for a given interval and number of Newton iterations.

Keywords Homomorphic encryption, Newton, CKKS, Division, Square root, Benchmark

1 Introduction

With the rise of digitalization, vast amounts of sensitive data are being generated, particularly in sectors such as healthcare, finance, and cloud computing [1]. At the same time, there is an increasing demand to analyze this data while complying with privacy laws and data protection regulations, such as the General Data Protection Regulation (GDPR)

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

in the European Union [2]. Traditional encryption methods effectively protect data during storage and transmission; however, they usually require decryption for processing. This requirement poses a security risk, as sensitive information may be exposed during the decryption stage. The numerous cloud data breaches in recent years highlight the reality of this threat. For example, in March 2026, attackers claimed to have successfully breached LexisNexis' cloud environment, resulting in the exfiltration of several gigabytes of structured data from its Amazon Web Services infrastructure [3]. Homomorphic encryption offers a compelling solution to this security problem, as it enables computations to be performed directly on encrypted data without decryption. This capability allows data to remain encrypted throughout its lifecycle in the cloud, making homomorphic encryption a promising solution for future data privacy.

However, existing fully homomorphic cryptosystems support only a limited set of operations out of the box—typically addition and multiplication. More complex functions, such as square root and division, must be constructed from these basic operations. The Newton-Raphson method is one approach to solve this problem. Prantl et al. [4] showed that it is the most suitable method for computing square root and inverse in the homomorphic cryptosystem CKKS.

This method requires suitable starting parameters to converge and yield meaningful results. For everyday PCs, square root and division operations are well-supported through standard libraries. However, this is not the case for the homomorphic cryptosystem CKKS: it does not natively support common auxiliary functions such as comparisons, bit shifts, and lookup tables, rendering existing methods largely unusable. This raises the key question: How can optimal initial parameters for the Newton–Raphson method be determined for computing roots and divisions in CKKS? To answer this question, we evaluate four initial parameter selection methods that are algorithmically compatible with CKKS: brute-force (the state-of-the-art from [4]), simulated annealing, hill climbing, and gradient descent.

In summary, the main contributions are: (1) Formalizing the selection of the Newton–Raphson initial value x_0 via $h(x) = mx + t$ as a black-box optimization problem. (2) A systematic comparison of three suitable optimization algorithms by evaluating over 60 300 problem instances. (3) Provide guidance for practioneers on the accuracy attainable for different interval sizes and multiplicative depths for homomorphic inverse and square root computation.

The remainder of this article is structured as follows: first, we introduce the background of Homomorphic Encryption (HE), the Newton-Raphson method, and the three optimization algorithms in Sect. 2. Next, we present related work in Sect. 3. Section 4 explains the implementation and parameters used in our optimization algorithms. Section 5 presents the results of these algorithms and compares them with the approach used by Prantl et al. [4]. Finally, we conclude the article in Sect. 7.

2 Background

We provide brief introductions to the core concepts used throughout this article. First, we define homomorphic cryptosystems, then present the Newton–Raphson method, and finally introduce the optimization algorithms used.

2.1 Homomorphic encryption

Homomorphic Encryption enables the evaluation of mathematical functions on encrypted data such that decrypting the resulting ciphertext yields the correct result, even when the party executing the operation does not know the contents of the ciphertexts. Analogous to [5, 6], we can formally extend an ordinary cryptosystem to be homomorphic as follows:

Definition 1 (*Cryptosystem*) We start with a cryptosystem defined as the five-tuple $\mathcal{P}, \mathcal{C}, \mathcal{K}, \mathcal{E}, \mathcal{D}$ where $\mathcal{P}$ is the set of plaintexts, $\mathcal{C}$ the set of ciphertexts, and $\mathcal{K}$ the set of all possible keys. $\mathcal{E}$ and $\mathcal{D}$ will be the encryption and decryption functions respectively. We define the encryption function $\mathcal{E}$ in combination with a key $k \in \mathcal{K}$ as $\mathcal{E}_k : \mathcal{P} \to \mathcal{C}$ and the decryption function $\mathcal{D}_k : \mathcal{C} \to \mathcal{P}$ where $\forall p \in \mathcal{P} : \mathcal{D}_k(\mathcal{E}_k(p)) = p$ holds true [7].

Definition 2 (*(Fully) homomorphic cryptosystem*) To make this cryptosystem homomorphic, we require both the plaintext and ciphertext spaces to form rings with two operations $\oplus$ and $\otimes$, yielding $(\mathcal{P}, \oplus_p, \otimes_p)$ and $(\mathcal{C}, \oplus_c, \otimes_c)$. The resulting structure is considered a fully homomorphic scheme if $\forall a, b \in \mathcal{P}$ and $k \in \mathcal{K}$: [8]

$$\mathcal{E}_k(a) \oplus_c \mathcal{E}_k(b) = \mathcal{E}_k(a \oplus_p b)$$
$$\mathcal{E}_k(a) \otimes_c \mathcal{E}_k(b) = \mathcal{E}_k(a \otimes_p b)$$

2.2 Newton–Raphson

Software integer division can be computed using functional iteration methods such as the Newton–Raphson method [9]. Prantl et al. [4] found the Newton–Raphson method to be well suited for homomorphic computation, as it only requires multiplication and addition. The universal iteration formula for the Newton method is given by Eq. (1).

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)} \tag{1}$$

Prantl et al. [4] proposed calculating the division a/b by computing the reciprocal of the divisor b and multiplying it by the dividend a. As the Newton–Raphson method is a zero-finding method, it requires a function $g(x)$ that equals 0 at $1/b$. A suitable function $g(x)$ and the corresponding iteration step x_{n+1} for the inverse are given by Eqs. (2) and (3).

$$g(x) = \frac{1}{x} - b \tag{2}$$

$$x_{n+1} = x_n \cdot (2 - x_n \cdot b) \tag{3}$$

This method can also be applied to calculate the square root of a. The corresponding function $g(x)$ and iteration step x_{n+1} for the inverse square root are given by Eqs. (4) and (5). After several iterations, Eq. (5) yields $1/\sqrt{a}$, which can be multiplied by a to obtain $\sqrt{a}$.

$$g(x) = \frac{1}{x^2} - a \tag{4}$$

$$x_{n+1} = x_n \cdot (1.5 - \frac{a}{2} \cdot x_n^2) \tag{5}$$

Both approximations require choosing a good starting value x_0 to converge [10]. Prantl et al. [4] use a homomorphically calculated linear helper function tailored to a predefined input interval to determine x_0 based on the input value itself. Given a ciphertext $\hat{x}$ in a known range $\hat{x} \in [a, b]$, the helper function h calculates the start value $h(\hat{x}) = c_0 + c_1 x = x_0$. Here, c_0, c_1 are parameters of the helper function. This step adds one level of multiplicative depth and a single addition to the homomorphic calculation. In practice, c_0 and c_1 are calculated in plaintext, given the expected interval, and used as ciphertext-plaintext operations during the homomorphic calculation of the linear approximation. A user needs to generate these parameters for an interval where encrypted values are expected to lie within. In total this approach consumes a multiplicative depth of $2n + 1$ and $3n + 2$ for the inverse and square root functions respectiveley.

2.3 Optimization algorithms

For our evaluation, we selected three well-known optimization algorithms, described in the following paragraphs based on Russell and Norvig [11].

2.3.1 Gradient descent (GD)

GD is a numerical optimization approach applicable to differentiable problems. It is an iterative method that takes small steps along the steepest-descent direction. Due to its greedy, gradient-based nature, this algorithm can only find global minima on convex functions.

2.3.2 Hill climbing (HC)

HC commonly refers to a family of similar heuristic local search algorithms. Essentially, HC algorithms follow a greedy approach by selecting a new state in the neighborhood of the current state that is better (with respect to a predefined metric) than the current one. Either a new state in the direction of greatest ascent is chosen, or a random but better state (*Stochastic Gradient Descent*). It is also possible to choose a new state at random and accept it only if it is better than the current one (*First-Choice Hill Climbing*). Yet another variant is called *Random-restart Hill Climbing*, where multiple hill climbers are started from different random initial states. These algorithms are also not guaranteed to find global maxima in non-convex problems.

2.3.3 Simulated annealing (SA)

SA is a heuristic optimization algorithm that can escape local maxima by occasionally accepting worse solutions. SA is inspired by the physical process of annealing metals, where materials are heated and gradually cooled to reach a lower-energy crystalline state. SA chooses a new state at random and always accepts it if it is better than the current state. However, even if the state is worse, it will be accepted with a certain probability $p = e^{\Delta E/T}$ (Boltzmann distribution), where ΔE is the change in state quality and T is the temperature that decreases gradually based on a predefined cooling schedule. As the temperature decreases, the probability of accepting worse states decreases in later iterations. Given a suitable cooling strategy, SA can converge to an optimal solution.

3 Related work

Prantl et al. [4] analyzed the suitability of various mathematical methods for basic mathematical functions in the context of HE. Because this technology is limited to a few operations, not every commonly used method performs well in this context. The Newton–Raphson method and Taylor series were found to be most suitable. To obtain initial values x_0 for the Newton–Raphson method, they use a linear approximation whose parameters were generated by a brute-force search. A range of search parameters is defined over the interval, and then each combination is tested, with the best selected. Although this approach yields good results, it has two drawbacks. First, the performance is very low since many configurations are tested. For large intervals, this process takes multiple hours. Second, optimal or at least near-optimal parameters must lie within the search space. However, it is not known a priori what such parameters can be, and small differences can have a large impact. This work aims to address the first problem and mitigate the second by employing optimization algorithms. The Newton–Raphson-based approach of [4] has also been applied to the homomorphic approximation of nonlinear activation functions [12], where the initial parameters are likewise determined by a brute-force search.

There is also work using the Newton–Raphson method to approximate the inverse function. Bonte et al. [13] use a fixed start value to approximate the inverse using the BFV [14, 15] cryptosystem. Ogilvie et al. [16] use a start line; however, their approach is limited to positive intervals only. Moon et al. [17] compare the performance of different approaches, such as Goldschmidt or Newton–Raphson, for calculating the inverse and inverse square root; however, they do not consider the choice of an initial guess. Instead, the intervals are chosen to be small to allow convergence. Wang et al. [18], the authors also calculate the square root by using Newton–Raphson's method to first approximate the inverse square root. To obtain a good starting approximation, they extend a technique proposed by Takagi [19]. The value is split into its most significant and least significant bits. A first-order Taylor series expansion is then calculated, where the required value for the most significant bits is obtained from a lookup table. Since lookup tables are difficult to use in the approximate cryptosystem CKKS [20] and are also not very efficient, we do not consider this approach practicable for HE. A lookup table is also used by Cornea-Hasegan et al. [21]. To calculate the square root of S, they also use the path via the inverse square root and use an approximate value of $1/\sqrt{S}$ as a starting point. Another well-known approach to calculate the inverse square root was first used in 1999 in the implementation of the first-person shooter Quake [22]. It is based on bit-shifting, where, at one point, a "magic" constant is subtracted, yielding a rough approximation to the inverse square root that can then be used to compute one or two iterations of Newton–Raphson's method to obtain an accurate result. There have been several modifications to this algorithm over the years, for example, by Walczyk et al. [23]. They used more constants and coefficients to achieve a more accurate solution. Since bit-shifting is not directly possible in the CKKS cryptosystem, we cannot consider this approach for homomorphic encryption either. There is also some literature with a direct focus on finding suitable start values for the (inverse) square root. One of them is by Kornerup and Muller [24]. They provide a formula that depends on the interval limits and the number of iterations and outputs a single starting value for all values in the interval. However, this approach does not scale for large intervals. Especially intervals that are

closer to zero cover a large span of different reciprocal square root values that cannot be approximated well by a single starting value. Such approaches are also presented in [10, 25]. Although they all aim to minimize the maximum approximation error, they cannot be used in homomorphic practice, as larger intervals are more common there.

Regarding division, there are also similar publications that use a lookup table [26, 27]. These approaches have the same problems regarding applicability in HE as the similar ones for the square root. Unlike the square root function, the inverse function is defined for both positive and negative values. If a lookup table is used, it can be simplified to contain only the values for one sign, as shown by Nenadic and Mladenovic [28]. However, in a homomorphic context, this presents an additional problem, as comparison based on the sign is both difficult and performance-intensive. Another approach for finding suitable start parameters stems from Rodrigues-Garcia et al. [29]. They use a piecewise second-degree polynomial approximation. Polynomials can be evaluated in HE; however, piecewise ones come with a performance penalty, as they require multiple comparisons. Using one polynomial for the whole interval also seems difficult, as the divergence towards $\pm\infty$ needs to be considered, and thus higher degrees are needed. This paper shows that straight lines already suffice.

Overall, the related work is primarily focused on hardware implementations, leveraging operations that are efficiently implemented. However, these are often not feasible or incur a large performance penalty in HE. Therefore, the view needs to be broadened to consider alternative methods for obtaining initial values, as is done in this work by using optimization algorithms to estimate straight-line coefficients.

4 Methods

To compute square root and division functions using the Newton–Raphson method, Prantl et al. [4] used a brute-force approach for finding suitable auxiliary functions $h(x) = mx + t$ to generate the required initial values homomorphically. However, using brute-force search to find m and t is computationally expensive and does not guarantee the optimal solution, since sampling an interval with infinite precision is not feasible. We therefore formulate this task as an optimization problem: for a given interval, find the two real coefficients of the auxiliary function that minimize the resulting approximation error. To address this task, we implement and analyze three widely used optimization algorithms: Gradient Descent (GD), a deterministic gradient-based local search, and two stochastic local search algorithms, Hill Climbing (HC) (first-choice Hill Climbing (HC) with random restarts) and Simulated Annealing (SA), the latter of which can additionally escape local minima.

As the success of the Newton–Raphson method strongly depends on a good initial guess (x_0), finding an auxiliary function that works for a whole interval is highly complex, and even slight deviations in the coefficients can lead to divergence. The same applies to optimization algorithms, which also require careful parameter selection to converge to meaningful results. These parameters also depend on the chosen evaluation intervals of the approximated functions. To obtain a broadly applicable method that works across a wide range of intervals without manual fine-tuning, we use several simple hyperparameter preselection strategies described in the following.

For all algorithms, we limit the maximum number of iterations to 50 000. Furthermore, all optimization algorithms require an objective function to determine the quality

of a given h. For this, we use the Root Mean Square Error (RMSE) calculated over 1000 samples uniformly distributed across the test interval. The objective function is shown in Eq. (6), where y denotes the ground truth and $\hat{y}$ the result of the Newton–Raphson approximation after k iterations.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=0}^{n}(y_i - \hat{y}_i)^2} \tag{6}$$

Gradient Descent (GD) We implemented GD with adaptive tuning of the learning rate, as shown in Algorithm 1. The fastest-converging learning rate in $\Gamma = \{1 \times 10^x \mid x \in [-16, 1]\}$ is determined and reduced by one order of magnitude to prevent an excessively large learning rate. Furthermore, the derivative required for GD is computed by calculating the central difference. As a final optimization, we apply early stopping if the gradient length is smaller than $1{\times}10^{-7}$.

Require: Objective function $J(\theta)$, learning rates $\Gamma = \{1 \times 10^x \mid x \in [-16, 1]\}$, Max Iterations $K_{max} = 50\,000$

Notation: e_i is the i-th standard basis vector; I is the identity matrix.

function TuneLearningRate(J, Γ)
    Find γ_{best} that minimizes $J(\theta)$ after 10 iterations $\forall \gamma \in \Gamma$
    return $\alpha \leftarrow \gamma_{best}/10$
end function

function GradientDescent($J, K_{max}, \alpha, \sigma^2$)
    $\alpha \leftarrow$ TuneLearningRate(J, Γ)
    Initialize $\theta \sim \mathcal{N}(0, \sigma^2 I)$
    for $k = 1$ to K_{max} **do**
        $g_i \approx \dfrac{J(\theta + \epsilon e_i) - J(\theta - \epsilon e_i)}{2\epsilon}$ ▷ Compute gradient with central difference
        if $\|g\|_2 < 10^{-7}$ **then**
            break ▷ Early stopping
        end if
        $\theta \leftarrow \theta - \alpha \cdot g$
    end for
    return θ
end function

Algorithm 1 Gradient Descent with Adaptive Tuning

Hill Climbing (HC) For the HC optimizer, we implemented a random-restart first-choice HC algorithm, as shown in Algorithm 2. We also include early stopping if the state quality does not improve over 500 consecutive iterations. The final result is selected from 10 parallel hill climbers, choosing the one with the best state quality.

Require: Objective function $J(\theta)$, restarts $R = 10$, patience $P = 500$, Max Iterations $K_{max} = 50\,000$

Notation: I is the identity matrix. Q denotes the state quality of J, here RMSE.
function HillClimber(J, P, K_{max})
    Initialize $\theta \sim \mathcal{N}(0, \sigma^2 I)$
    $stagnation \leftarrow 0$, $k \leftarrow 0$
    while $stagnation < P$ **and** $k < K_{max}$ **do**
        $\theta_{next} \leftarrow \mathcal{N}(\theta, 0.01I)$ ▷ Choose neighbor of θ
        if $J(\theta_{next}) < J(\theta)$ **then**
            $\theta \leftarrow \theta_{next}$
            $stagnation \leftarrow 0$
        else
            $stagnation \leftarrow stagnation + 1$
        end if
        $k \leftarrow k + 1$
    end while
    return θ
end function

function MultiHillClimbing(J, R, P, K_{max})
    $Q_{best} \leftarrow \infty, \theta_{final} \leftarrow$ null
    for $j = 1$ to R **do**
        $\theta_{tmp} \leftarrow$ HillClimber(J, P, K_{max})
        if $J(\theta_{tmp}) < Q_{best}$ **then**
            $Q_{best} \leftarrow J(\theta_{tmp})$
            $\theta_{final} \leftarrow \theta_{tmp}$
        end if
    end for
    return θ_{final}
end function

Algorithm 2 Random-Restart Hill Climbing

Simulated Annealing (SA) Our implementation of SA with quadratic cooling and an adaptive initial variance search is shown in Algorithm 3. We first select a suitable variance for choosing the neighboring states by trying $\sigma^2 \in \{10^{-15}, 10^{-14}, \ldots, 10^{-1}\}$ for 50 iterations. To ensure a stable learning environment, we select the largest σ^2 for which the state quality did not diverge and reduce it by one order of magnitude.

Require: Objective function $J(\theta)$, Max iterations K_{max}, Initial guess θ_0
Notation: I is the identity matrix. Q denotes the state quality of J, here RMSE.
function TuneVariance(J)
    Test $\sigma^2 \in \{10^{-15}, 10^{-14}, \ldots, 10^{-1}\}$ over 50 iterations
    Let σ^2_{max} be largest variance where $J(\theta) \neq \pm\infty$ ▷ ∞ referring to the numerical limits of C++ double
    return $\sigma^2 \leftarrow \sigma^2_{max}/10$
end function

function Anneal(J, K_{max}, θ_0)
    $\sigma^2 \leftarrow$ TuneVariance(J)
    $\theta_{global} \leftarrow\sim \mathcal{N}(0, \sigma^2 I)$, $\theta_{best} \leftarrow \theta_0$
    $Q_{global} \leftarrow J(\theta_0)$, $Q_{best} \leftarrow J(\theta_0)$
    $T \leftarrow 1$
    for $k = 1$ to K_{max} **do**
        $\theta_{local} \leftarrow \theta_{global} + \mathcal{N}(0, \sigma^2 I)$
        $Q_{local} \leftarrow J(\theta_{local})$
        $\Delta \leftarrow Q_{local} - Q_{global}$
        $P \leftarrow \exp(-\Delta/T)$
        if $Q_{local} < Q_{best}$ **then**
            $\theta_{best} \leftarrow \theta_{local}$, $Q_{best} \leftarrow Q_{local}$
        end if
        if $Q_{local} < Q_{global}$ **or** $\mathcal{N}(0,1) < P$ **then**
            $\theta_{global} \leftarrow \theta_{local}$, $Q_{global} \leftarrow Q_{local}$
        end if
        $T \leftarrow 1 - (\frac{k}{K_{max}})^2$ ▷ Quadratic cooling
    end for
    if $Q_{best} < Q_{global}$ **then** ▷ If there has been a better state during the search, take it instead
        $\theta_{global} \leftarrow \theta_{best}$
    end if
    return θ_{global}
end function

Algorithm 3 Simulated Annealing with Quadratic Cooling

Ultimately, we evaluated the computation of the division and square root functions on the same intervals and for the same number of iterations as Prantl et al. [4] to facilitate comparison with their results.

Choice of optimizers The choice of GD, HC and SA is motivated by the structure if the optimization problem. We seek to determine only two real-valued parameters, *m* and *t*, using a simple objective function: minimizing the RMSE over a given interval. This constitutes a low dimensional optimization problem, with a single solution. Therefore, simple to implement and low overhead optimization algorithms are a natural fit for this problem. The selected algorithms progressively increase in complexity. We begin with GD, a deterministic, gradient-based local search algorithm, followed by HC a stochastic local search algorithm, and finally SA, a stochastic local search algorithm capable of escaping local minima. More elaborate optimization algorithms such as population-based methods like genetic algorithms [30] or particle swarm optimization [31], generally require significantly higher computational overhead. This overhead is disproportionate for a low-dimensional optimization problem; therefore, these methods were excluded from the study.

Table 1 Cryptographic parameters used for evaluation; naming according to OpenFHE documentation

Parameter	Value
MultDepth	30
ScaleModSize	59
BatchSize	32 768
ScalingTechnique	FIXEDAUTO
SecretKeyDist	UNIFORM_TERNARY

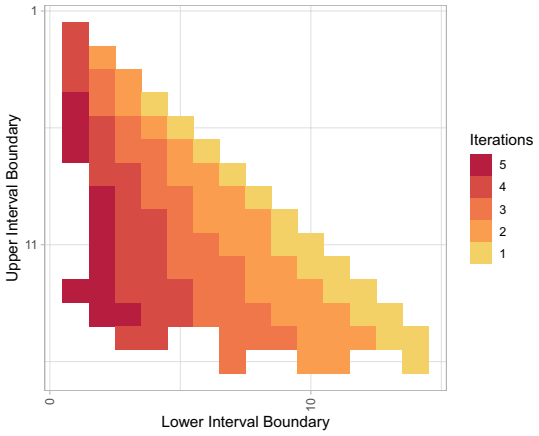


Fig. 1 Minimum number of iterations required to achieve 95% accuracy criterion. SQRT function with GD optimizer

5 Evaluation

To obtain conclusive results, we evaluated each optimization algorithm for the division and square root functions over 20 100 distinct intervals using one to five Newton iterations, yielding a total of 603,000 evaluated configurations.

5.1 Experimental setup

For homomorphic evaluation we used the OpenFHE Library [32] in version 1.4.2. The libraries parameters are shown in Table 1, remaining parameters are automatically chosen by OpenFHE. The parameters were chosen to represent a typical setting for basic homomorphic applications. Furthermore, we reused the same cryptographic context and keys for all measurements, to rule out the influence varying parameters or seeds may have on the results. The experiments were done on a machine with a AMD EPYC 9354P processor with 128GB of main memory running Ubuntu 24.04 LTS (Linux 6.8.0-101-generic).

5.2 Square root

To evaluate the square root function, we generated parameters with all three optimizers for all valid intervals $I = \{[a, b] \mid 0 \leq a < b \leq 200\}$ with the number of Newton iterations $i_{sqrt} \in [1, 5]$. The intervals themselves were sampled at 0.1-step increments and encrypted into a single ciphertext; the function is then evaluated over the whole interval at once by exploiting the Single Instruction, Multiple Data (SIMD) capability of CKKS ciphertexts. To enable a comparison with [4], we additionally determine, for each interval, the smallest number of Newton iterations such that 95% of the y values lie within an ϵ-environment of the true value ($|y - y_{true}| \leq 0.1$). Figures 1, 2, and 3 visualize the

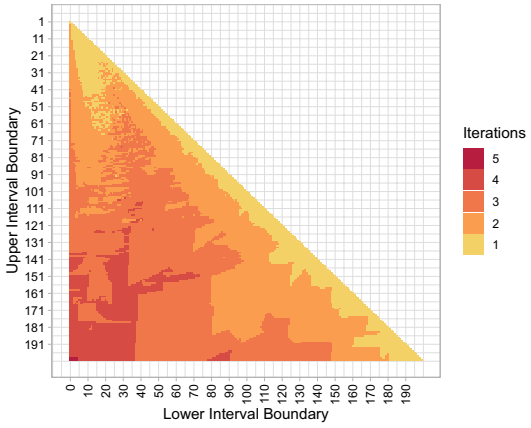


Fig. 2 Minimum number of iterations required to achieve 95% accuracy criterion. SQRT function with HC optimizer

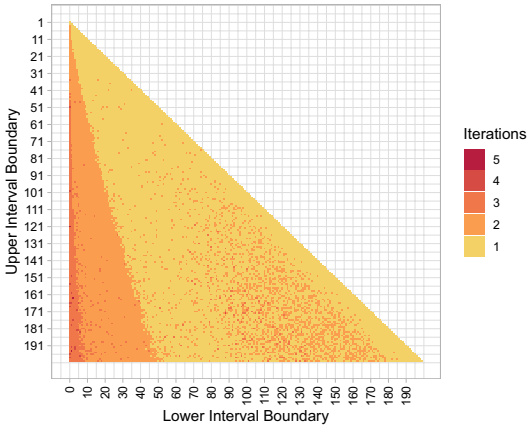


Fig. 3 Minimum number of iterations required to achieve 95% accuracy criterion. SQRT function with SA optimizer

smallest number of Newton iterations required to achieve this accuracy criterion for GD, HC, and SA, respectively. GD failed to produce valid parameters for 99.1–99.6% of the 20,100 test intervals per iteration count (see Table 2), and for the remaining intervals it requires the highest iteration counts, particularly for intervals starting at 0. In contrast, HC in Fig. 2 generates parameters that enable the computation of the square root on all test intervals with at most 5 iterations. SA, illustrated in Fig. 3, was also able to generate parameters for all intervals and typically requires a maximum of only 3 Newton iterations. Only for isolated intervals up to 5 iterations are necessary. The variation in the required iteration count across intervals indicates that parameter generation is not optimal for every interval; as analyzed in Sect. 5.5, this variation is consistent with the run-to-run variance inherent to stochastic optimization, and can be expected to be reduced by additional fine-tuning of SA parameters such as the cooling schedule, or by running multiple SA instances with different seeds. The original Fig. 4 published by Prantl et al. [4] shows that the brute-force parameters evaluate most intervals with at most 10 iterations, while intervals close to 0 could not be evaluated with fewer than 10 iterations. Comparing SA and HC with these results shows that the use of optimization algorithms substantially reduces the number of required Newton iterations, since both optimizers satisfy the accuracy criterion on all test intervals with at most 5 iterations. For homomorphic square root evaluation, this translates into a corresponding reduction

Table 2 Metrics summary of the tested optimizers for SQRT (best $RMSE_{\%}$ in bold)

i_{sqrt}	Optimizer	$RMSE_{\%}$	$RMSE_{mean}$	$RMSE_{min}$	$RMSE_{max}$	#NA
1	GD	0.0	1.00e−03	0e+00	3.0500e−01	19,919
2	GD	0.0	1.00e−03	0e+00	2.1500e−01	19,946
3	GD	0.0	4.90e−02	0e+00	1.5600e−01	19,974
4	GD	0.0	9.70e−02	0e+00	6.7820e+00	19,997
5	GD	0.0	0.00e+00	0e+00	8.4000e−02	20,014
1	HC	6.4	4.74e−01	0e+00	2.2810e+00	0
2	HC	8.6	1.13e−01	0e+00	7.7300e−01	0
3	HC	12.1	1.30e−02	0e+00	3.0200e−01	0
4	HC	22.7	1.00e−03	0e+00	9.9000e−02	0
5	HC	40.5	0.00e+00	0e+00	2.6000e−02	0
1	SA	**93.6**	6.20e−02	0e+00	1.0694e+01	0
2	SA	**91.4**	1.30e−02	0e+00	4.2880e+00	0
3	SA	**87.9**	4.00e−03	0e+00	1.5097e+01	0
4	SA	**77.3**	1.00e−03	0e+00	4.3880e+00	0
5	SA	**59.5**	2.00e−03	0e+00	1.7733e+01	0

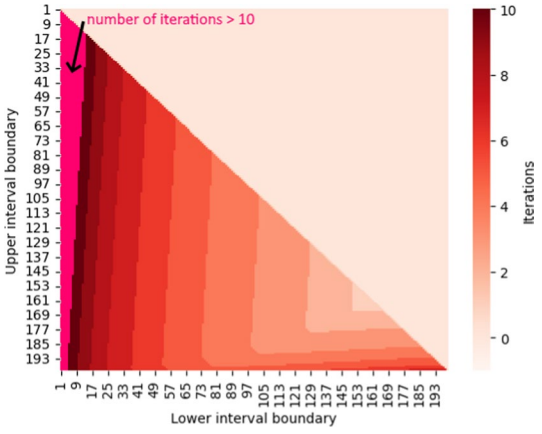


Fig. 4 Minimum number of iterations required to achieve 95% accuracy criterion. SQRT function with brute-force optimizer

of the multiplicative depth: with the brute-force parameters of [4], an interval requiring $i_{sqrt} \geq 10$ Newton iterations performs $3 \cdot i_{sqrt} + 2 \geq 32$ consecutive multiplications, whereas with the optimized parameters the maximum multiplicative depth over all test intervals is $3 \cdot 5 + 2 = 17$; using SA, a single Newton iteration suffices for most intervals, requiring only a maximum of $3 \cdot 1 + 2 = 5$ consecutive multiplications. Overall, the optimization-based approach saves up to 27 consecutive multiplications, depending on the evaluation interval.

Since the absolute tolerance $\epsilon = 0.1$ of the ϵ-environment is relatively loose for small intervals, and the 95% quantile tolerates up to 5% arbitrarily large outliers, we also analyzed the results with respect to the RMSE, Mean Square Error (MSE), and Mean Absolute Error (MAE). Figures 5 and 6 show the overall accuracy (RMSE) for 3 and 4 iterations (SA and HC respectively). For the majority of intervals the RMSE is at most 1×10^{-10}, and some intervals reach 1×10^{-13} with only 4 Newton iterations, indicating that the optimized parameters have no large outliers within these intervals.

Summarizing the mean, minimum, and maximum accuracy over all test intervals for different Newton iterations i_{sqrt} is shown in Table 2. We also include the number of

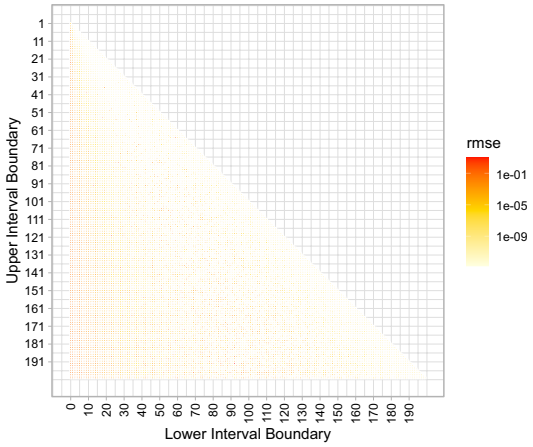


Fig. 5 SQRT with SA and $i_{SQRT} = 3$ (RMSE)

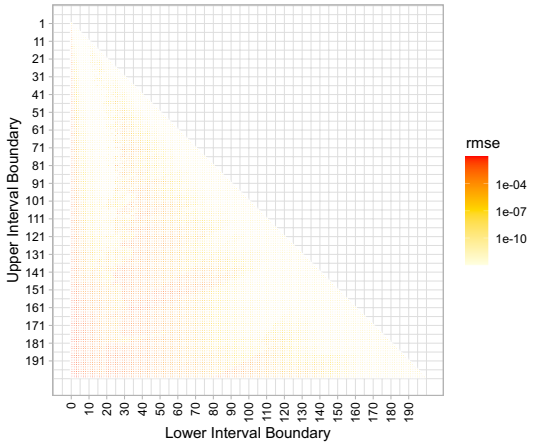


Fig. 6 SQRT with HC and $i_{SQRT} = 4$ (RMSE)

times the evaluation failed due to suboptimal parameters or failure in encrypted evaluation (#NA). A #NA can happen due to the optimizer returning initial line parameters, that do not yield a converging initial guess for certain interval parts or a decryption failure due to CKKS approximation errors, which can grow, due to subobtimal initial parameters. $RMSE_{\%}$ represents the percentage of intervals, where an optimization algorithm performed the best out of the three with respect to RMSE. As already visible in the previous figures, 19 919 to 20 014 of the 20 100 test intervals (99.1–99.6%) per iteration count are not evaluable with GD while the #NA counts for HC and SA are zero: every interval and iteration count converged to a meaningful result and was successfully evaluated homomorphically. In terms of worst-case error, HC bounds the worst RMSE by 2.5×10^{-2}, whereas the worst RMSE of SA is higher, reaching ≈ 17.7. In terms of the $RMSE_{\%}$ metric, SA achieves the lowest RMSE on 93.6% down to 59.5% of the test intervals for $i_{sqrt} \in [1, 5]$, i.e., on the largest share of intervals for every iteration count, while HC does so on only 6.4% up to 40.5% of the intervals and GD on none. HC provides, on average, slightly worse but more stable results.

Further analysis of the suitability of the tested optimizers for different intervals can be inferred from Figs. 7, 9 and 11 for $i_{sqrt} = \{1, 3, 5\}$ respectively. The figures show, for each optimizer, the fraction of the test intervals of a given length on which it has the lowest

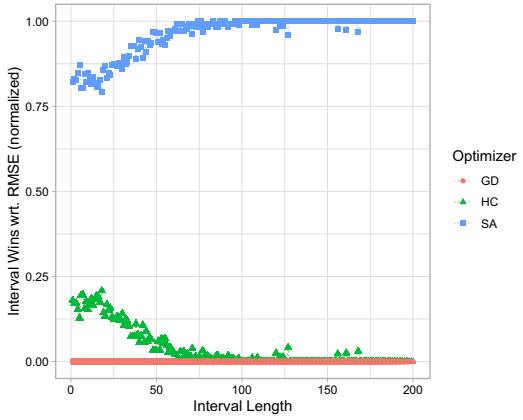


Fig. 7 SQRT normalized wins per optimizer for $i_{sqrt} = 1$

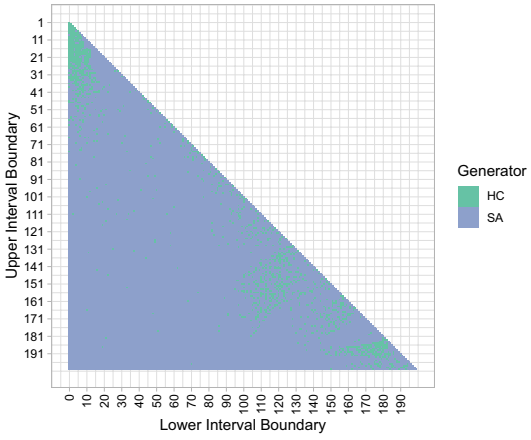


Fig. 8 Best optimizer for SQRT w.r.t. rmse for $i_{sqrt} = 1$

RMSE (we refer to such an interval as a "win"), normalized by the number of intervals of that length. From these figures, we can infer that SA has the lowest RMSE on the majority of the test intervals for every value of i_{sqrt}, while HC achieves the lowest RMSE on an increasing share of the shorter intervals for higher i_{sqrt}. As the previous results have already shown, GD achieves the lowest RMSE on no interval at all and is therefore not suitable for finding an auxiliary start-value function for the square root function. Figures 8, 10 and 12 show, for $i_{sqrt} = \{1, 3, 5\}$, the optimizer with the lowest RMSE for each interval. In case of equal RMSE, the optimizers are sorted in alphabetical order, giving GD followed by HC a slight edge over SA in the few undecided cases. The heatmaps are dominated by SA, followed by HC, which confirms that GD is not competitive in any configuration.

5.3 Division

To evaluate the division function, we generated parameters with all three optimizers for all valid intervals $I = \{[a, b] \mid -100 \leq a < b \leq 100\}$ with the number of Newton iterations $i_{inv} \in [1, 5]$. The intervals themselves were sampled at 0.1-step increments and encrypted as described in Sect. 5.2. As in [4], we determine, for each interval, the smallest number of Newton iterations such that 95% of the y values lie within the ϵ

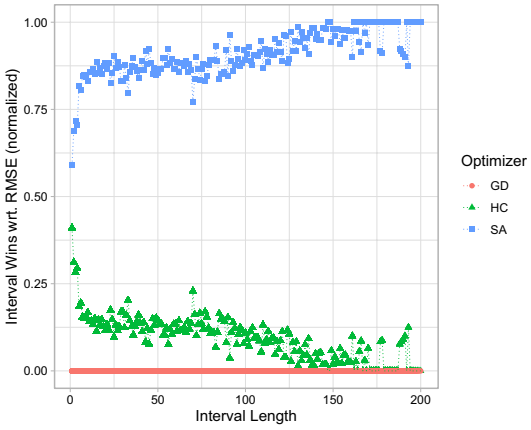


Fig. 9 SQRT normalized wins per optimizer for $i_{sqrt} = 3$

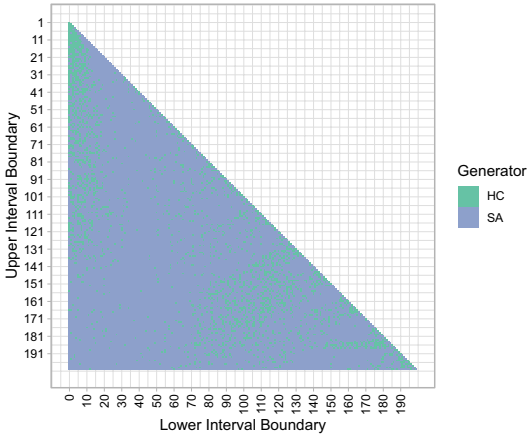


Fig. 10 Best optimizer for SQRT w.r.t. rmse for $i_{sqrt} = 3$

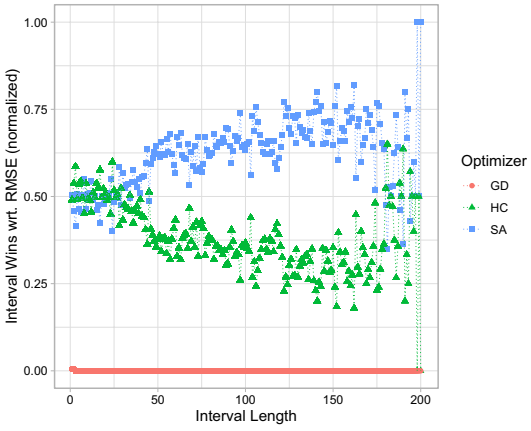


Fig. 11 SQRT normalized wins per optimizer for $i_{sqrt} = 5$

-environment. Figures 13, 14, and 15 show these results for GD, HC, and SA, respectively, while Fig. 16 shows the corresponding results for the brute-force parameters of [4] for comparison. All three optimizers produce parameters for which intervals containing only negative or only positive numbers satisfy the accuracy criterion, with most



Fig. 12 Best optimizer for SQRT w.r.t. rmse for $i_{sqrt} = 5$

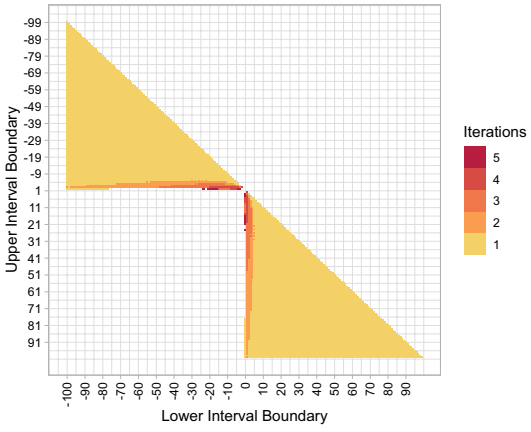


Fig. 13 Minimum number of iterations required to achieve 95% accuracy criterion. INV function with GD optimizer

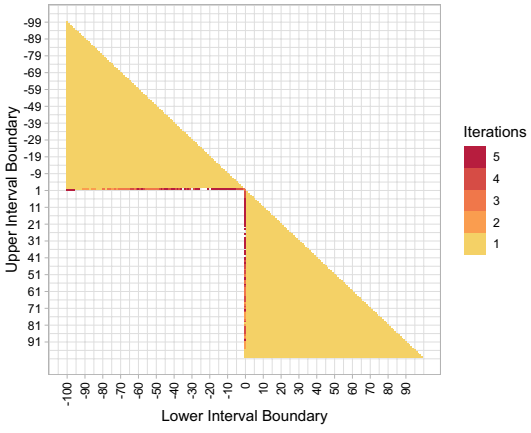


Fig. 14 Minimum number of iterations required to achieve 95% accuracy criterion. INV function with HC optimizer

of these intervals achieving it with a single Newton iteration. Intervals containing 0 are harder to approximate, because the inverse function is unbounded near the zero. The mean RMSE over all intervals stays between 0.31 and 0.35 for $i_{inv} \in [1, 5]$, and the worst RMSE reaches 3.58 for GD and 3.42 for HC and SA (Table 3). Similar to the square root,

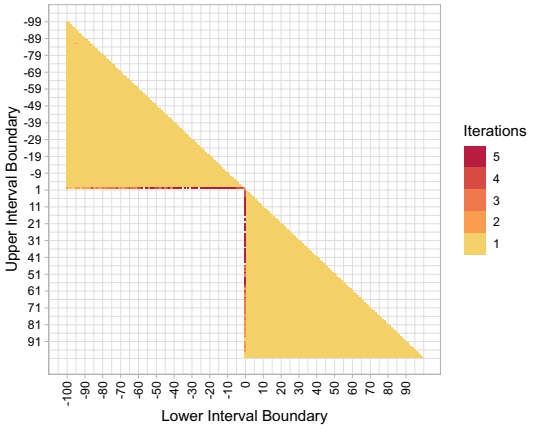


Fig. 15 Minimum number of iterations required to achieve 95% accuracy criterion. INV function with SA optimizer

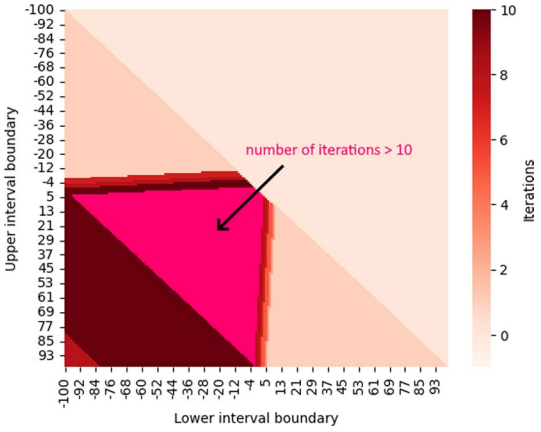


Fig. 16 Minimum number of iterations required for 95% of values to be contained in the ϵ-environment

Table 3 Metrics summary of the tested optimizers for INV (best $RMSE_{\%}$ in bold)

i_{inv}	Optimizer	$RMSE_{\%}$	$RMSE_{mean}$	$RMSE_{min}$	$RMSE_{max}$	#NA
1	GD	**45.3**	3.23e−01	0e+00	3.579e+00	0
2	GD	48.7	3.18e−01	0e+00	3.320e+00	76
3	GD	28.1	3.10e−01	0e+00	2.937e+00	293
4	GD	9.0	3.31e−01	0e+00	2.595e+00	10,622
5	GD	3.2	3.49e−01	0e+00	2.309e+00	15,984
1	HC	42.1	3.18e−01	0e+00	3.423e+00	0
2	HC	13.3	3.16e−01	0e+00	3.182e+00	0
3	HC	11.7	3.14e−01	0e+00	2.836e+00	0
4	HC	24.3	3.12e−01	0e+00	2.595e+00	0
5	HC	28.0	3.10e−01	0e+00	2.309e+00	0
1	SA	21.9	3.18e−01	0e+00	3.423e+00	0
2	SA	**56.1**	3.16e−01	0e+00	3.182e+00	0
3	SA	**75.2**	3.14e−01	0e+00	2.836e+00	0
4	SA	**71.4**	3.12e−01	0e+00	2.595e+00	0
5	SA	**70.9**	3.10e−01	0e+00	2.309e+00	0

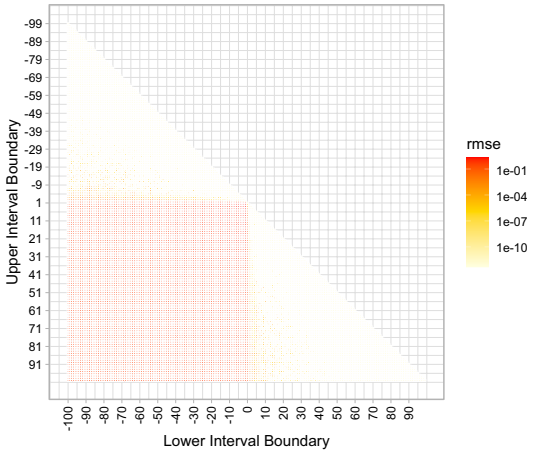


Fig. 17 INV with SA and $i_{INV} = 5$ (RMSE)

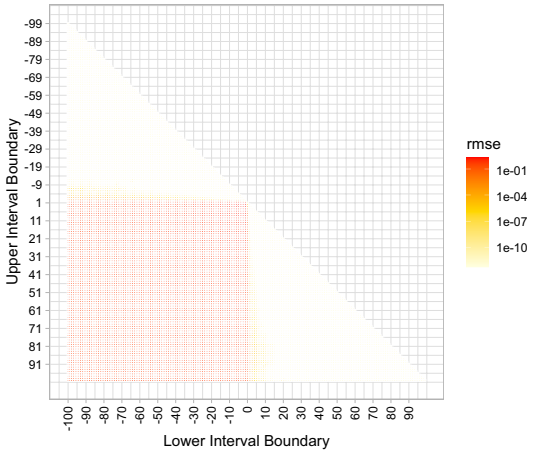


Fig. 18 INV with HC and $i_{INV} = 5$ (RMSE)

SA attains the lowest RMSE on the largest share of the intervals (see Table 3); in particular, it does so on the difficult intervals around 0 for $i_{inv} \in \{3, 5\}$ (Figs. 21 and 23). For the intervals that start or end at 0, the optimized parameters satisfy the same accuracy criterion as in [4] but with only 1 to 5 iterations in most cases, instead of ≥ 10, corresponding to a saving of 10 to 18 consecutive multiplications and hence to shallower homomorphic circuits.

Figures 17 and 18 show the RMSE for 5 Newton iterations with the SA and HC algorithms, respectively. Most intervals not containing 0 achieve an RMSE below 1×10^{-10} with 5 iterations for both optimizers, whereas the intervals containing 0 yield an RMSE of ≈ 0.5.

Table 3 shows the minimum, maximum, and mean RMSE values over all measured intervals for any given number of iterations. The value $RMSE_{\%}$ shows the percentage of intervals, where an optimizer achieved the lowest RMSE for any given number of iterations. Furthermore, the #NA column shows the number of intervals, where the parameters returned by the optimizer could not be used to successfully approximate the inverse function and the evaluation failed. GD failed to produce evaluable parameters for 10 622 of the 20 100 test intervals (52.8%) for $i_{inv} = 4$ and for 15 984 (79.5%) for $i_{inv} = 5$, with

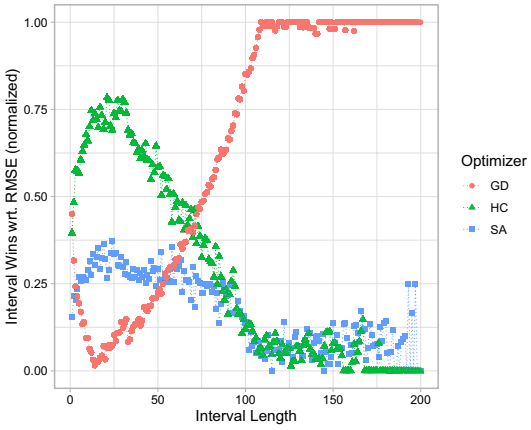


Fig. 19 INV normalized wins per optimizer for $i_{inv} = 1$

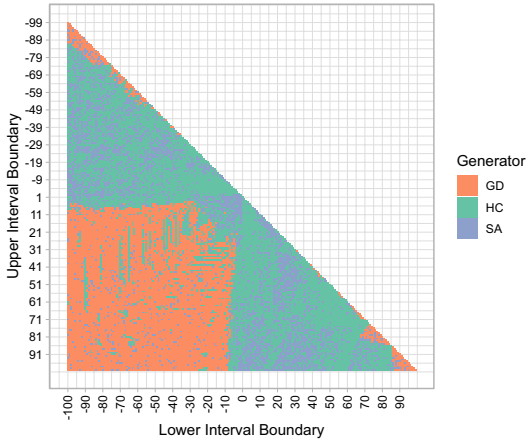


Fig. 20 Best optimizer for INV w.r.t. rmse for $i_{inv} = 1$

the failure rate growing from 0 (0%) at $i_{inv} = 1$ to 79.5% at $i_{inv} = 5$. In contrast, HC and SA have #NA counts of zero, i.e., they produce valid parameters for all interval–iteration combinations. In terms of the $RMSE_{\%}$ metric, SA has the lowest RMSE on 56.1% to 75.2% of the intervals for $i_{inv} \in [2, 5]$, while for $i_{inv} = 1$ it is GD, which has the lowest RMSE on 45.3% of the intervals, ahead of HC (42.1%) and SA (21.9%).

Because the inverse function varies substantially across intervals, especially when 0 is contained in the interval, we examined its performance across various intervals. To assess the suitability of the optimization algorithms for various interval lengths, we determined, for each interval length, the fraction of the intervals of that length on which an optimizer has the lowest RMSE; we refer to such an interval as a "win" of the respective optimizer. The results are shown in Figs. 19, 21 and 23 for $i_{inv} \in \{1, 3, 5\}$, respectively. Since intervals with a length ≥ 100 include 0, a distinct change of the behavior is visible for those intervals. For $i_{inv} = 1$, GD has the lowest RMSE on most of the intervals larger than 100, while HC has the lowest RMSE on most of the shorter intervals. For $i_{inv} \in \{3, 5\}$, SA has the lowest RMSE on most of the intervals, independent of their length. Figures 20, 22, and 24 show the optimizer with the lowest RMSE for each interval for $i_{inv} \in \{1, 3, 5\}$, respectively, and confirm this distribution. Comparing these figures with Table 3, we again conclude that SA achieves the lowest RMSE on the largest share

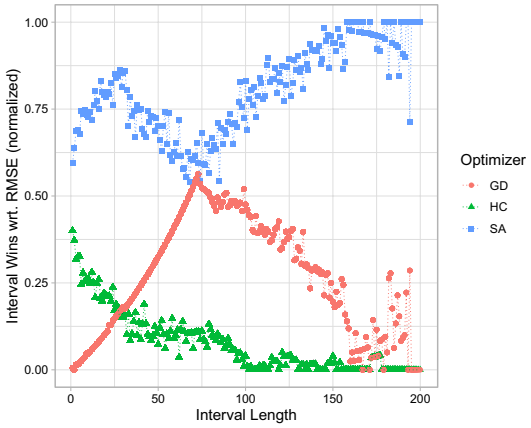


Fig. 21 INV normalized wins per optimizer for $i_{inv} = 3$

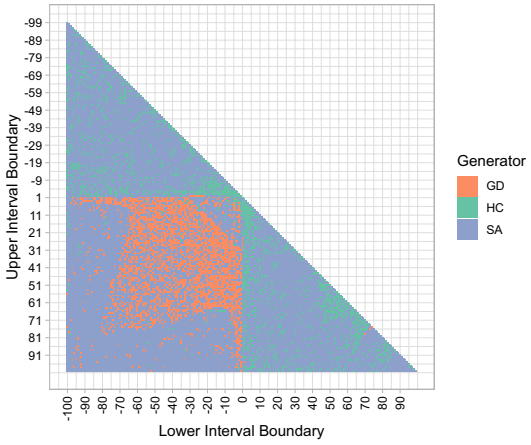


Fig. 22 Best optimizer for INV w.r.t. rmse for $i_{inv} = 3$

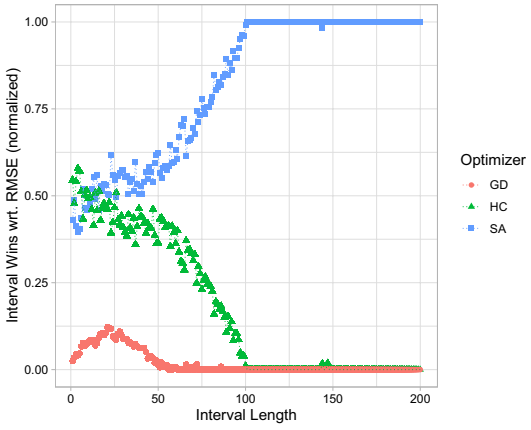


Fig. 23 INV normalized wins per optimizer for $i_{inv} = 5$

of the intervals across a wide range of interval lengths and Newton iterations, while HC also achieves an RMSE of $\approx 1\times^{-10}$ with 5 iterations on most intervals not containing 0.

Furthermore, this once again demonstrates that the Newton method with the linear approximation for x_0 is not well-suited for computing division when 0 lies within the

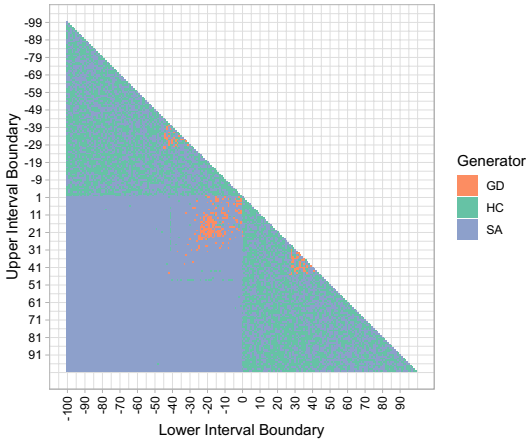


Fig. 24 Best optimizer for INV w.r.t. rmse for $i_{inv} = 5$

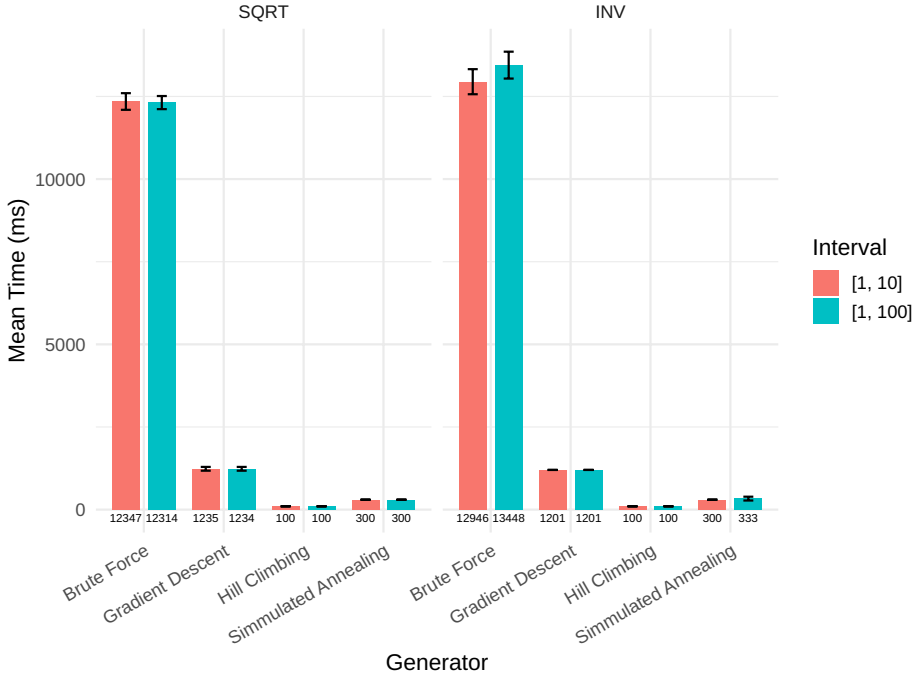


Fig. 25 Mean parameter generation times for different intervals, generators, and functions

interval. Therefore, a different helper function or a method to handle the sign separately is required to perform division more accurately on these intervals.

5.4 Optimizer performance

In this section we analyze the runtime of the optimization algorithms and brute-force (BF). Figure 25 shows the mean parameter generation time for the different optimization algorithms and brute-force over 30 runs and $i = 2$ for the intervals $I_{eval} \in \{[1, 10], [1, 50], [1, 100]\}$. The brute-force approach was limited to the search interval $c_0, c_1 \in [-1, 1]$ which was sampled in 0.01 and 0.0001 steps for the intercept and slope, respectively. This configuration results in $201 \cdot 20,001 = 4,020,201$ evaluations of all 1000 samples of the interval. While this is only a snapshot, finer sampling or larger search intervals would increase the runtime further. Since an equal number of samples is

used for parameter generation, runtime depends only on the generator, not on the interval size or the function itself. Furthermore, the runtime between runs is quite consistent. The GD, HC, and SA algorithms were limited to at most 50,000 iterations (500,000 for HC considering 10 parallel hill climbers), which also represents the upper bound on objective evaluations of the interval. This example demonstrates that the optimization algorithms outperform the brute-force search by two to three orders of magnitude when it comes to runtime, while simultaneously yielding parameters with lower RMSE and lower multiplicative depth than brute-force.

5.5 Optimizer variance

As the optimization algorithms depend on random variables, we measured the run-to-run variation for 30 different random seeds. Figure 26 shows three intervals $I_{eval} \in \{[1, 10], [1, 50], [1, 100]\}$ for the inverse and square-root functions with $i = 2$ iterations each with the min, max and mean measured accuracies. While GD shows no variation the SA and HC optimizers show some variation in the resulting RMSE. The SA optimizers run-to-run variation is mainly scattered towards higher values, while the HC algorithm run-to-run variation is more scattered around the mean, which gets larger for larger intervals. In practice, this means that one could run SA multiple times with different random seeds or with slightly adjusted hyperparameters (e.g., slower cooling, different cooling series) to potentially obtain a slightly more accurate initial helper function.

6 Discussion

The evaluation shows that optimization algorithms for finding the initial guess can improve the accuracy of Newton-based evaluations of the inverse and square root functions, or enable fewer iterations to achieve similar accuracy as [4]. Exact execution times for the optimization algorithms depend on their hyperparameters, such as sampling rates and number of iterations, and vary with the chosen configuration. We used the RMSE for the objective function metric, however the optimization algorithms can in principle also be used with other error metrics such as the maximum error. We emphasize that, in practice, one must account for noise introduced by CKKS, depending on the chosen cryptographic parameters, as well as accumulated noise from prior computations. The results of this article thus serve as a reference guide for developers to verify whether the Newton-based approach is feasible for a given application, to determine the expected accuracy for a given configuration, and to select reasonable parameters for achieving the desired accuracy. As motivated in Sect. 4, we deliberately restrict the comparison to GD, HC and SA. While there exist other optimization algorithms like genetic algorithms, particle swarm optimization, differential evolution or Bayesian optimization, they are unlikely to improve the initial parameters substantially over SA while requiring significantly higher computational cost. Whether these excluded methods could outperform SA or HC on specific intervals therefore remains an open question for future work.

7 Conclusion

In this work we formalized the selection of the parameters of a linear initial helper function for Newton–Raphson-based homomorphic square root and division evaluation as a black-box optimization problem, and we systematically evaluated three representative optimizers: Gradient Descent (GD), Hill Climbing (HC), and Simulated Annealing (SA).

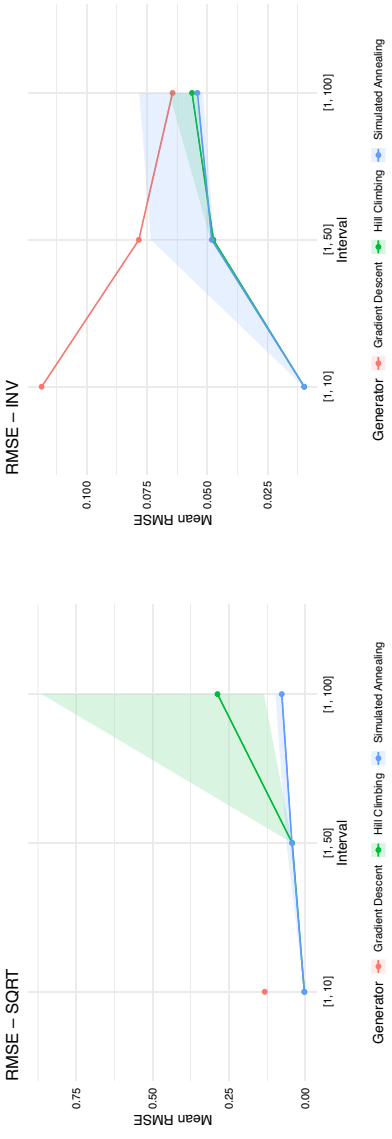


Fig. 26 Mean, min and max final accuracy for 30 runs with varying seeds

Our extensive empirical study covers over 603 000 configurations across a wide range of intervals and iteration counts. The results show that SA achieves the lowest RMSE on the largest share of intervals: 93.6% down to 59.5% for one to five Newton iterations on the square root function, and 56.1% to 75.2% for two to five iterations on the inverse function (Tables 2 and 3). In several configurations, however, HC attains a lower RMSE, establishing it as a viable alternative. In contrast, GD is not suited for the non-convex error landscape of the optimization problem: for the square root function it fails to produce valid parameters for 99.1–99.6% of the 20 100 test intervals, and for the inverse function the share of intervals without valid parameters grows from 0% for a single Newton iteration to 79.5% for 5 iterations.

Overall, we have demonstrated that the Newton–Raphson approach to homomorphic square root and division can be substantially improved by using optimization to find a better starting guess. The optimized helpers deliver comparable accuracy with substantially fewer Newton iterations or equivalently, higher accuracy for a fixed iteration budget. Therefore we were able to reduce the number of consecutive multiplications by a factor of 2 and 3 per saved iteration for the inverse and square root functions, respectively. Moreover, finding a suitable helper function now takes a few hundred milliseconds instead of seconds to minutes, which is a significant advantage for practical applications.

Combining the strengths of the SA and HC approaches into a joint parameter generation scheme is a promising direction for covering wide ranges of intervals. In future work, we aim to mitigate the run-to-run variability of the SA approach and to extend the method to additional helper functions, such as higher-degree polynomials, in order to further improve homomorphic evaluation, particularly of division. We also plan to apply this approach to other Newton-based homomorphic evaluations, such as the approximation of activation functions.

Author contributions
S.E. led all aspects of the research, including experimental setup, data collection, and manuscript preparation. All authors provided technical support and assisted with manuscript revisions. All authors have read and agreed to the published version of the manuscript.

Funding
The authors declare that no financial support was received for the research, authorship, and/or publication of this article.

Data availability
All data generated by this study are included in this article.

Declarations

Ethics approval and consent to participate
Not applicable.

Consent for publication
Not applicable.

Clinical trial number
Not applicable.

Competing interests
The authors confirm that there are no financial or non-financial conflict of interest.

Received: 13 March 2026 / Accepted: 28 September 2026
Published online: 22 October 2026

References

1. Thallam NST. Privacy-preserving data analytics in the cloud Leveraging homomorphic encryption for big data security. 2021. https://doi.org/10.5281/ZENODO.16418272.
2. Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA relevance); 2016.
3. Priya A. LexisNexis data breach: threat actor claims theft of 2.04 GB of data. CyberPress. https://cyberpress.org/lexisnexis-data-breach/. Accessed 05 Mar 2026.
4. Prantl T, Horn L, Engel S, Iffländer L, Beierlieb L, Krupitzer C, et al. De bello homomorphico: investigation of the extensibility of the openfhe library with basic mathematical functions by means of common approaches using the example of the ckks cryptosystem. Int J Inf Secur. 2023. https://doi.org/10.1007/s10207-023-00781-0.
5. Prantl T, Horn L, Engel S, Bauer A, Kounev S. Quo vadis ckks: Comparison of the realization of basic mathematical functions for the homomorphic cryptosystem ckks using de bello and polynomial approximations. J Inf Secur Appl. 2025;93:104110. https://doi.org/10.1016/j.jisa.2025.104110.
6. Prantl T, Schneider T, Horn L, Engel S, Iffländer L, Krupitzer C. Oratio homomorphico: creating a lecture for homomorphic encryption by applying design-based research. In: Proceedings of the 4th EAI conference on computer science and education in computer science. EAI CSECS 2025. New York: Springer; 2025.
7. Buchmann J. Introduction to cryptography (undergraduate texts in mathematics). New York: Springer; 2004. p. 335.
8. Yi X, Paulet R, Bertino E. Homomorphic Encryption and Applications. Springer; 2014. https://doi.org/10.1007/978-3-319-12229-8.
9. Rodeheffer T. Software integer division. Technical Report MSR-TR-2008-141; 2008. https://www.microsoft.com/en-us/research/publication/software-integer-division/.
10. Montuschi P, Mezzalama M. Optimal absolute error starting values for Newton-Raphson calculation of square root. Computing. 1991;46(1):67–86. https://doi.org/10.1007/bf02239012.
11. Russell SJ. Artificial intelligence. Boston Always Learning, 4th edn., global. edn. Pearson, Boston; 2022.
12. Engel S, Prantl T, Horn L, Bonilla R, Kounev S. Homomorphic approximation of non-linear activation functions using the ckks cryptosystem of the openfhe library. Discover Artif Intell. 2026. https://doi.org/10.1007/s44163-026-01915-8.
13. Bonte C, Vercauteren F. Privacy-preserving logistic regression training. BMC Med Genom. 2018;11(4):86. https://doi.org/10.1186/s12920-018-0398-y.
14. Brakerski Z. Fully homomorphic encryption without modulus switching from classical gapsvp. In: Safavi-Naini R, Canetti R, editors. Advances in cryptology—CRYPTO 2012. Berlin, Heidelberg: Springer; 2012. p. 868–86.
15. Fan J, Vercauteren F. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Paper 2012/144; 2012. https://eprint.iacr.org/2012/144.
16. Ogilvie T, Player R, Rowell J. Improved privacy-preserving training using fixed-Hessian minimisation. Cryptology ePrint Archive, Paper 2020/1514; 2020. https://eprint.iacr.org/2020/1514.
17. Moon J, Omarov Z, Yoo D, An Y, Chung H. Adaptive successive over-relaxation method for a faster iterative approximation of homomorphic operations. Cryptology ePrint Archive, Paper 2024/1366; 2024. https://eprint.iacr.org/2024/1366.
18. Wang LK, Schulte MJ. Decimal floating-point square root using Newton-Raphson iteration. In: IEEE international conference on application-specific systems, architecture processors (ASAP'05). 2005. pp. 309–15. https://doi.org/10.1109/ASAP.2005.29.
19. Takagi N. Powering by a table look-up and a multiplication with operand modification. IEEE Trans Comput. 1998;47(11):1216–22. https://doi.org/10.1109/12.736432.
20. Cheon JH, Kim A, Kim M, Song Y. Homomorphic encryption for arithmetic of approximate numbers. In: Takagi T, Peyrin T (eds) Advances in cryptology—ASIACRYPT. Cham: Springer; 2017. pp. 409–437. https://doi.org/10.1007/978-3-319-70694-8_15.
21. Cornea-Hasegan MA, Golliver RA, Markstein P. Correctness proofs outline for Newton–Raphson based floating-point divide and square root algorithms. In: Proceedings 14th IEEE symposium on computer arithmetic (Cat. No.99CB36336); 1999. pp. 96–105.https://doi.org/10.1109/ARITH.1999.762834.
22. id Software: Quake-III-Arena/Code/Game/Q_math.c at Master Id-Software/Quake-III-Arena; 1999. https://github.com/id-Software/Quake-III-Arena/blob/master/code/game/q_math.c.
23. Walczyk CJ, Moroz LV, Cieśliński JL. A modification of the fast inverse square root algorithm. Computation. 2019;7(3):41. https://doi.org/10.3390/computation7030041.
24. Kornerup P, Muller J-M. Choosing starting values for certain Newton-Raphson iterations. Theoret Comput Sci. 2006;351(1):101–10. https://doi.org/10.1016/j.tcs.2005.09.056.
25. Moursund DG. Optimal starting values for Newton-Raphson calculation of $\sqrt{x}\frac{1}{2}$. Commun ACM. 1967;10(7):430–2. https://doi.org/10.1145/363427.363454.
26. Louvet N, Muller JM, Panhaleux A. Newton–Raphson algorithms for floating-point division using an FMA. In: ASAP 2010—21st IEEE international conference on application-specific systems, architectures and processors; 2010. pp. 200–207. https://doi.org/10.1109/ASAP.2010.5540948.
27. Wang LK, Schulte MJ. Decimal floating-point division using Newton-Raphson iteration. In: Proceedings. 15th IEEE international conference on application-specific systems, architectures and processors. 2004. pp. 84–95. https://doi.org/10.1109/ASAP.2004.1342461.
28. Nenadic NM, Mladenovic SB. Fast division on fixed-point DSP processors using Newton–Raphson method. In: EUROCON 2005—the international conference on "Computer as a Tool", vol. 1; 2005. pp. 705–708. https://doi.org/10.1109/EURCON.2005.1630028.
29. Rodriguez-Garcia A, Pizano-Escalante L, Parra-Michel R, Longoria-Gandara O, Cortez J. Fast fixed-point divider based on Newton-Raphson method and piecewise polynomial approximation. In: 2013 International conference on reconfigurable computing and FPGAs (ReConFig); 2013. pp. 1–6. https://doi.org/10.1109/ReConFig.2013.6732291.
30. Mirjalili S. Genetic algorithm. In: Evolutionary algorithms and neural networks: theory and applications. Cham: Springer; 2018. pp. 43–55.

31. Wang D, Tan D, Liu L. Particle swarm optimization algorithm: an overview. Soft computing. 2018;22(2);387–408. https://doi.org/10.1007/s00500-016-2474-6.
32. Badawi AA, Alexandru A, Bates J, Bergamaschi F, Cousins DB, Erabelli S, Genise N, Halevi S, Hunt H, Kim A, Lee Y, Liu Z, Micciancio D, Pascoe C, Polyakov Y, Quah I, RV, S, Rohloff K, Saylor J, Suponitsky D, Triplett M, Vaikuntanathan V, Zucca V. OpenFHE: open-source fully homomorphic encryption library. Cryptology ePrint Archive, Paper 2022/915; 2022. https://eprint.iacr.org/2022/915.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.